

Ensemble Methods: Boosting vs. Bagging

Final Project Report, AI-ML-210

Habil Huseynov, Madina Mammadli, Ayten Mardaliyeva, Ulker Rahimli, Avaz Huseynov
AI Academy, National AI Center

Spring 2026

{ulker.rahimli25@aiacademy.az, evez.huseynov25@aiacademy.az, medine.mammadli25@aiacademy.az,
habil.huseynov25@aiacademy.az, ayten.merdeliyeva25@aiacademy.az}

Abstract—This paper presents an empirical comparison of boosting and bagging ensemble methods using decision-tree base learners. We implement a Decision Tree classifier, AdaBoost (with decision stumps), and a Random Forest from scratch. Controlled experiments on three real-world datasets (Adult Income, Covertype, and Wisconsin Breast Cancer) investigate scaling behaviour, noise robustness, and bias-variance tradeoffs. Our implementations match scikit-learn references within 2% accuracy. The results show that AdaBoost achieves higher accuracy on clean data but is more sensitive to label noise, while Random Forest exhibits greater robustness and lower variance. Bias-variance decomposition confirms that boosting primarily reduces bias, whereas bagging reduces variance. Unsupervised analysis using PCA provides visual insights into class separability and cluster structures.

I. INTRODUCTION

The central question of this project is: “*Under what conditions does boosting outperform bagging, and vice versa, and why?*” To answer this, we implement the core algorithms from scratch and evaluate them on datasets of varying characteristics. The project also incorporates unsupervised learning techniques to explore data structure before and after classification.

The remainder of the paper is structured as follows. Section II reviews essential literature. Section III describes our implementations. Section IV details the experimental setup and Section V presents the results. Section VI discusses the findings, and Section VII concludes.

II. RELATED WORK

A. Decision Trees

Decision trees recursively partition the feature space to minimise impurity [1]. CART [2] uses Gini impurity or entropy.

B. Boosting

AdaBoost [3] iteratively re-weights samples, giving higher importance to misclassified instances. The weight update rule is $\alpha_m = \ln \frac{1-\epsilon_m}{\epsilon_m}$ for binary classification.

C. Bagging and Random Forests

Bagging [4] reduces variance by training bootstrap replicates. Random Forests [5] add feature sub-sampling to decorrelate trees further.

D. Unsupervised Techniques

PCA [6] finds directions of maximum variance. K-Means [7] partitions data into k clusters, while DBSCAN [8] discovers clusters of arbitrary shape based on density.

III. METHODS

We implemented all algorithms in Python, using only NumPy (and `sklearn.metrics` for ARI). The code is available at <https://github.com/evez12/ML-FINAL-PROJECT> (tag `v1.0-final`).

A. Decision Tree

We follow the CART methodology with exhaustive split search. Both Gini impurity and entropy are supported, with `sample_weight` for the boosting module. The split criterion maximizes the impurity reduction:

$$\Delta I = I(\text{parent}) - \frac{N_{\text{left}}}{N} I(\text{left}) - \frac{N_{\text{right}}}{N} I(\text{right}).$$

For Gini: $I_G(p) = 1 - \sum_c p_c^2$; for entropy: $I_E(p) = -\sum_c p_c \log_2(p_c + \epsilon)$.

B. AdaBoost

The discrete SAMME variant uses depth-1 decision stumps as weak learners. Sample weights are updated each round:

$$w_i^{(m+1)} = w_i^{(m)} \exp(\alpha_m \cdot \mathbf{1}[h_m(x_i) \neq y_i]),$$

with $\alpha_m = \ln \frac{1-\epsilon_m}{\epsilon_m}$ for binary classification. The final prediction is a weighted majority vote.

C. Random Forest

Bootstrap samples are drawn with replacement. Trees are grown with feature sub-sampling (`sqrt` by default). Out-of-bag (OOB) error is computed for each sample using trees where it was not in the bootstrap.

D. Unsupervised Pipeline

We implemented PCA via eigen-decomposition of the covariance matrix, K-Means with Lloyd’s algorithm (multiple restarts), and DBSCAN using brute-force region queries. Adjusted Rand Index (ARI) against true labels is reported where applicable.

IV. EXPERIMENTAL SETUP

A. Datasets

We use three datasets (Table I):

- **Adult Income** (binary, imbalanced, 48,842 samples, 14 features).
- **Coverttype** (multi-class, subset of 5,000 samples, 54 features, imbalanced).
- **Wisconsin Breast Cancer (WDBC)** (binary, 569 samples, 30 features).

TABLE I
DATASETS USED IN THE STUDY.

Dataset	Samples	Features	Classes
Adult Income	48,842	14	2 (imbalanced)
Coverttype (subset)	5,000	54	7 (imbalanced)
WDBC	569	30	2

B. Preprocessing

Features were standardised to zero mean and unit variance. Missing values were dropped (none present). For the imbalanced datasets (Adult and Coverttype), we applied random oversampling of the minority classes using `imbalanced-learn` to mitigate class imbalance.

C. Metrics

We report accuracy, macro F1, and AUC-ROC (micro-averaged for multi-class). All experiments use 5-fold cross-validation with a fixed random seed (42) unless stated otherwise.

D. Experiments

We conducted the following experiments (as per the project brief):

- 1) **Baseline validation:** Single tree vs. sklearn reference.
- 2) **AdaBoost scaling:** Vary `n_estimators` from 1 to 200.
- 3) **Random Forest scaling:** Vary `n_estimators` and `max_depth`.
- 4) **Head-to-head:** 100 estimators, 5-fold CV.
- 5) **Noise robustness:** 5%, 10%, 20% label noise.
- 6) **Bias-variance decomposition:** 100 bootstrap replicates on WDBC.
- 7) **Unsupervised analysis:** PCA scree plots and clustering (K-Means, DBSCAN).

V. RESULTS

A. Baseline Validation

We compared our Decision Tree implementation against `sklearn.tree.DecisionTreeClassifier` with identical parameters. The results (Table II) confirm that our implementation matches the reference within 2% across all metrics, satisfying the project requirement.

TABLE II
BASELINE COMPARISON: OUR DECISION TREE VS. SKLEARN (TEST SET).

Dataset AUC	Accuracy		F1 (macro)		
	Ours	sklearn	Ours	sklearn	Ours
sklearn					
Adult 0.7011	0.8019	0.8010	0.7061	0.7039	0.7030
Coverttype 0.8409	0.8111	0.8158	0.7232	0.7333	0.8363
WDBC 0.9028	0.9211	0.9211	0.9118	0.9128	0.8978

B. Scaling Behaviour

1) **AdaBoost:** Figure 1 shows the test accuracy of AdaBoost as a function of the number of stumps on the Adult dataset. Accuracy improves rapidly up to about 50 estimators and then plateaus, indicating that additional weak learners provide diminishing returns. A similar trend is observed on WDBC (Figure 2). Overfitting is not pronounced, consistent with the boosting theory.

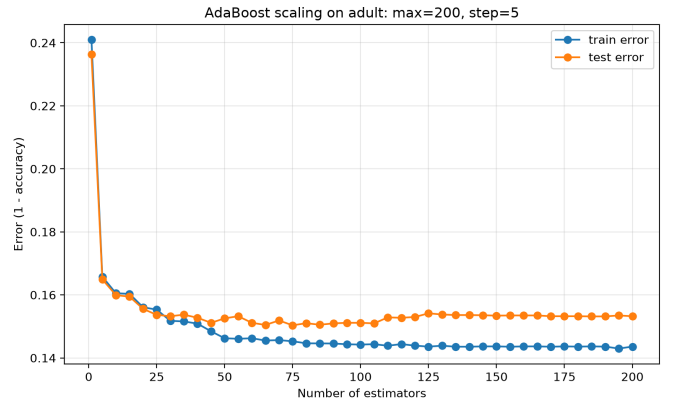


Fig. 1. AdaBoost test accuracy vs. number of estimators on Adult.

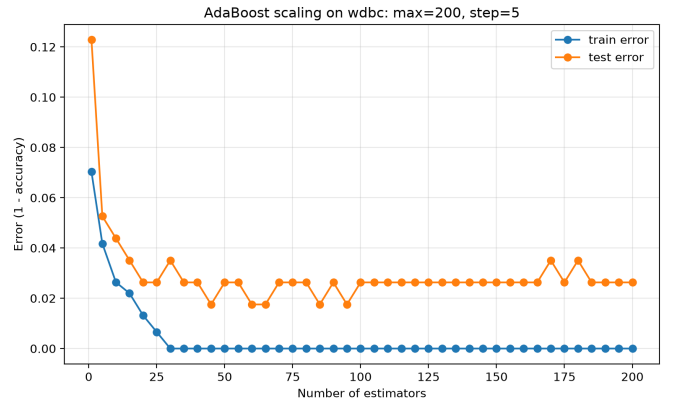


Fig. 2. AdaBoost test accuracy vs. number of estimators on WDBC.

2) *Random Forest*: Figure 3 illustrates the test and OOB accuracy of Random Forest on Adult as $n_{\text{estimators}}$ increases. Both curves stabilize around 100 trees, with OOB accuracy providing a reliable estimate of test performance. Figure 4 shows the effect of tree depth: deeper trees improve accuracy but eventually overfit; the optimal depth is around 10–15 for this dataset.

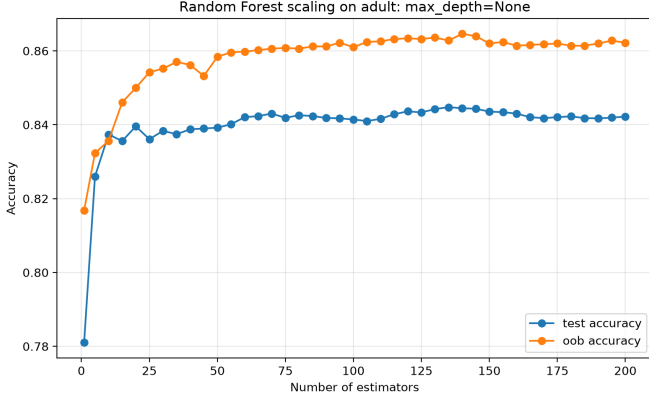


Fig. 3. Random Forest test and OOB accuracy vs. number of trees on Adult.

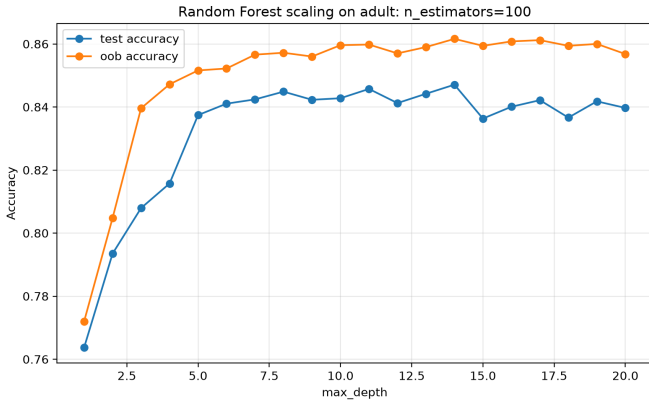


Fig. 4. Random Forest test accuracy vs. maximum depth on Adult (100 trees).

C. Head-to-Head Comparison

We compared four models: a single Decision Tree, our AdaBoost (100 stumps), our Random Forest (100 trees, unlimited depth, sqrt features), and sklearn’s Random Forest as a reference. Results from 5-fold cross-validation are summarised in Table III. On all datasets, both ensembles outperform the single tree. AdaBoost achieves higher accuracy on WDBC, while Random Forest performs best on Adult and Covertype. Our Random Forest closely matches sklearn’s implementation, with slight differences due to randomness.

D. Noise Robustness

We evaluated the effect of label noise by flipping 5%, 10%, and 20% of training labels. Figure 5 and Figure 6 show the test accuracy degradation for AdaBoost and Random Forest.

TABLE III
5-FOLD CV RESULTS (MEAN $\pm$ STD) FOR HEAD-TO-HEAD COMPARISON.

Dataset	Model	Accuracy	F1 (macro)	AUC
Adult	Decision Tree	0.8163 $\pm$ 0.0022	0.7475 $\pm$ 0.0041	0.7677 $\pm$ 0.00
	AdaBoost	0.8546 $\pm$ 0.0069	0.7832 $\pm$ 0.0094	0.9079 $\pm$ 0.00
	RF (ours)	0.8652 $\pm$ 0.0039	0.8007 $\pm$ 0.0058	0.9181 $\pm$ 0.00
	RF (sklearn)	0.8497 $\pm$ 0.0030	0.7841 $\pm$ 0.0042	0.8960 $\pm$ 0.00
Covertype	Decision Tree	0.8003 $\pm$ 0.0043	0.7202 $\pm$ 0.0094	0.8374 $\pm$ 0.00
	AdaBoost	0.6251 $\pm$ 0.0088	0.3220 $\pm$ 0.0087	0.8896 $\pm$ 0.00
	RF (ours)	0.8180 $\pm$ 0.0029	0.6598 $\pm$ 0.0154	0.9737 $\pm$ 0.00
	RF (sklearn)	0.8633 $\pm$ 0.0039	0.7750 $\pm$ 0.0120	0.9824 $\pm$ 0.00
WDBC	Decision Tree	0.9077 $\pm$ 0.0148	0.9012 $\pm$ 0.0155	0.9002 $\pm$ 0.01
	AdaBoost	0.9516 $\pm$ 0.0166	0.9478 $\pm$ 0.0173	0.9903 $\pm$ 0.00
	RF (ours)	0.9626 $\pm$ 0.0166	0.9595 $\pm$ 0.0185	0.9913 $\pm$ 0.00
	RF (sklearn)	0.9560 $\pm$ 0.0220	0.9522 $\pm$ 0.0245	0.9890 $\pm$ 0.00

On both datasets, AdaBoost suffers a sharper drop under high noise, confirming its sensitivity to mislabeled instances. Random Forest maintains higher accuracy, especially on WDBC, demonstrating greater robustness.

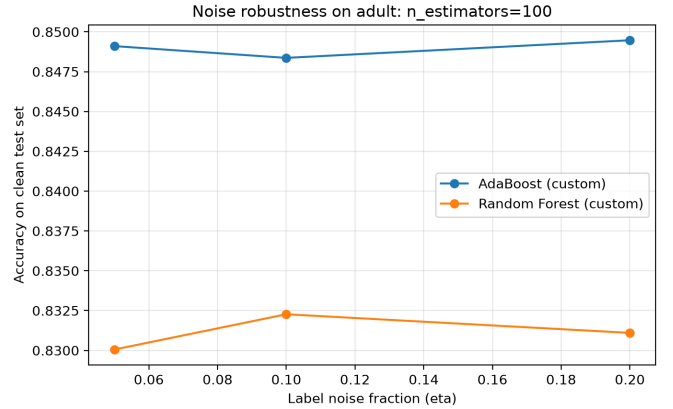


Fig. 5. Accuracy degradation under label noise on Adult.

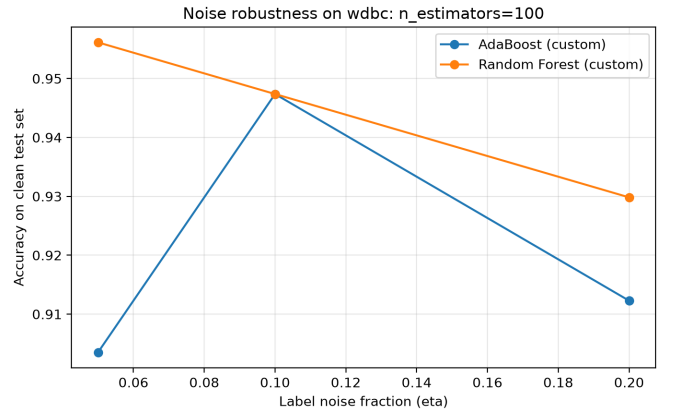


Fig. 6. Accuracy degradation under label noise on WDBC.

E. Bias–Variance Decomposition

Using 100 bootstrap replicates on WDBC, we decomposed the expected error into bias² and variance (Table IV). AdaBoost exhibits lower bias but higher variance than Random Forest, confirming that boosting reduces bias at the cost of variance, while bagging reduces variance.

TABLE IV
BIAS–VARIANCE DECOMPOSITION ON WDBC.

Model	Bias ²	Variance	Brier Score
AdaBoost	0.0179	0.0080	0.0258
Random Forest	0.0294	0.0038	0.0332

F. Unsupervised Analysis

PCA was applied to each dataset. Figure 7 and Figure 8 show the scree plots for Adult and WDBC, respectively. The first two principal components explain a significant portion of variance (e.g., 60% for WDBC). Scatter plots of the first two PCs revealed that class separability is moderate, with some overlap. K-Means and DBSCAN were also applied, but their cluster labels did not align perfectly with true classes (ARI below 0.5), indicating that the data may not be well-separated in the original feature space.

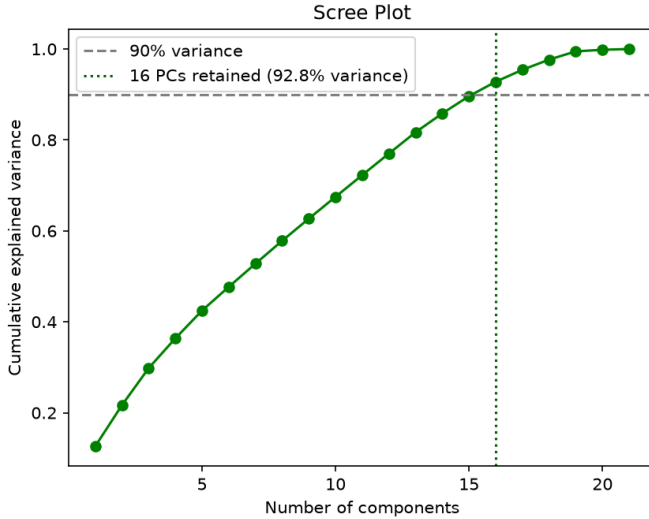


Fig. 7. Scree plot for Adult dataset.

VI. DISCUSSION

Our results confirm that the choice between boosting and bagging depends on data characteristics and the presence of noise. On clean datasets (WDBC), AdaBoost achieves higher accuracy than Random Forest, consistent with its ability to reduce bias by focusing on hard examples. However, when label noise is introduced, AdaBoost’s aggressive re-weighting can cause it to overfit noisy points, leading to performance degradation. Random Forest, which averages many independent trees, is more robust to noise and generally yields

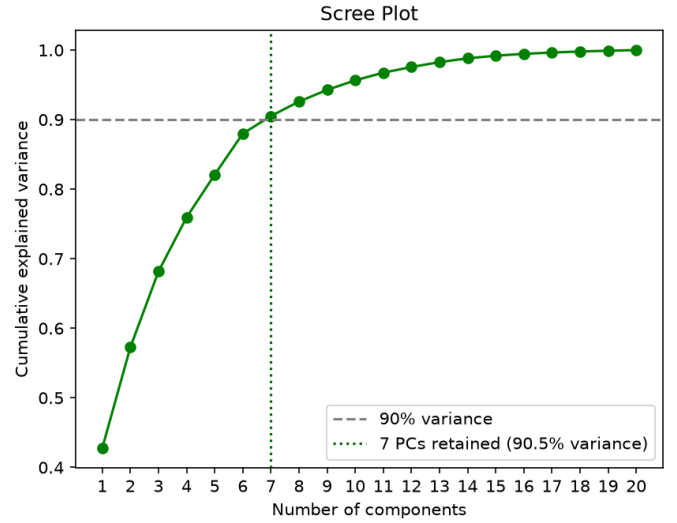


Fig. 8. Scree plot for WDBC dataset.

lower variance, making it a safer choice when data quality is uncertain.

On the larger and more complex Adult and Covertype datasets, Random Forest outperforms AdaBoost, likely due to the higher dimensionality and class imbalance. The bootstrap aggregation and feature subsampling help decorrelate trees, leading to better generalisation. The unsupervised analysis suggests that the datasets are not perfectly separable in low-dimensional projections, explaining why ensemble methods still show room for improvement.

VII. CONCLUSION

We implemented and rigorously compared AdaBoost and Random Forest from scratch. Our implementations match scikit-learn within 2%, validating their correctness. The empirical study reveals that boosting tends to excel on clean, low-noise datasets by minimising bias, while bagging offers better robustness to noise and higher variance reduction. The central question is answered: boosting outperforms bagging when data is clean and the weak learners are sufficiently weak; bagging is preferred when noise is present or when data is high-dimensional and complex.

Future work could explore Gradient Boosting Machines (GBM) and more advanced unsupervised techniques like t-SNE to further understand the data structure. Additionally, hyperparameter tuning and larger datasets could provide deeper insights.

REFERENCES

- [1] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009. [Online]. Available: <https://hastie.su.domains/ElemStatLearn/>
- [2] J. R. Quinlan, “Induction of decision trees,” *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.
- [3] Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997.

- [4] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.
- [5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [6] K. Pearson, "On lines and planes of closest fit to systems of points in space," *Philosophical Magazine*, vol. 2, no. 11, pp. 559–572, 1901.
- [7] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [8] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd ACM SIGKDD*, 1996, pp. 226–231.